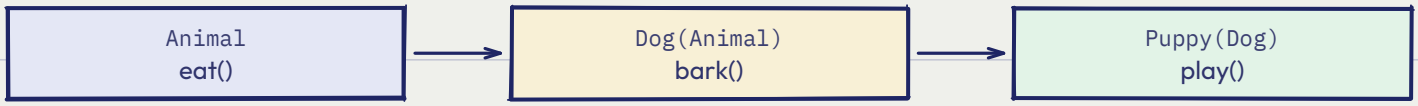
INHERITANCE: REUSE CODE

Inheritance lets a child class reuse a parent class. Syntax: `class Child(Parent):`

1) Single and multilevel inheritance



Example :

```
class Animal:
    def eat(self):
        print("Animal eats")
class Dog(Animal):
    def bark(self):
        print("Dog barks")
class Puppy(Dog):
    def play(self):
        print("Puppy plays")
puppy1 = Puppy()
puppy1.eat()
puppy1.bark()
puppy1.play()
```

Output :

```
Animal eats
Dog barks
Puppy plays
```

Puppy never defines eat() or bark(), it inherits both through the chain

2) Multiple inheritance: many parents

Example :

```
class Teacher:
    def teach(self):
        print("Teacher teaches")
class Researcher:
    def research(self):
        print("Researcher researches")
class Professor(Teacher, Researcher):
    def guide(self):
        print("Professor guides students")
professor1 = Professor()
professor1.teach()
professor1.research()
professor1.guide()
```

Output :

```
Teacher teaches
Researcher researches
Professor guides students
```

__INIT__ AND SUPER()

A child can have its own `__init__()`. Use `super().__init__(...)` to reuse the parent's setup.

1) Employee and Manager

Example :

```
class Employee:
    def __init__(self, name, age, salary, department):
        self.name = name
        self.age = age
        self.salary = salary
        self.department = department
    def personal_details(self):
        print(f"Name: {self.name}, Age: {self.age}")
        print(f"Salary: {self.salary}, Department: {self.department}")
class Manager(Employee):
    def __init__(self, name, age, salary, department, team_size):
        super().__init__(name, age, salary, department)
        self.team_size = team_size
    def show_team(self):
        print(f"Team size: {self.team_size}")
manager1 = Manager("Amit", 32, 70000, "Finance", 8)
manager1.personal_details()
manager1.show_team()
```

Output :

```
Name: Amit, Age: 32
Salary: 70000, Department: Finance
Team size: 8
```

☆ WHY SUPER()?

- The parent already knows how to set up `name`, `age`, `salary` and `department`.
- `super().__init__(...)` reuses that code instead of writing it again.

2) Multilevel with super()

Example :

```
class Employee:
    def __init__(self, name, salary):
        self.name = name
        self.salary = salary
class Manager(Employee):
    def __init__(self, name, salary, team_size):
        super().__init__(name, salary)
        self.team_size = team_size
class SeniorManager(Manager):
    def __init__(self, name, salary, team_size, department):
        super().__init__(name, salary, team_size)
        self.department = department
senior1 = SeniorManager("Amit", 80000, 10, "IT")
print(senior1.name, senior1.salary, senior1.team_size, senior1.department)
```

Output :

```
Amit 80000 10 IT
```

POLYMORPHISM & ABSTRACTION

Polymorphism means one method name, different behaviour. **Abstraction** hides the details.

1) Method overriding

Example :

```
class Animal:
    def sound(self):
        print("Animal makes a sound")
class Dog(Animal):
    def sound(self):
        print("Dog barks")
class Cat(Animal):
    def sound(self):
        print("Meow")
Animal().sound()
Dog().sound()
Cat().sound()
```

Output :

```
Animal makes a sound
Dog barks
Meow
```

2) Same call, different classes

Example :

```
class EmailAlert:
    def send(self):
        print("Sending an email")
class SMSAlert:
    def send(self):
        print("Sending an SMS")
for alert in [EmailAlert(), SMSAlert()]:
    alert.send()
```

Output :

```
Sending an email
Sending an SMS
```

3) Abstraction: ABC and @abstractmethod

Example :

```
from abc import ABC, abstractmethod
class Vehicle(ABC):
    @abstractmethod
    def start(self):
        pass
class Car(Vehicle):
    def start(self):
        print("Car starts with a key")
Car().start()
```

Output :

```
Car starts with a key
```

children must implement the abstract method; Vehicle() itself cannot be created

PRACTICE & REVISION

Write each class yourself first, then compare. Each solution is run for real.

Task 1 Single inheritance.

Solution :

```
class Person:
    def introduce(self):
        print("I am a person")
class Student(Person):
    def learn(self):
        print("I am learning Python")
s = Student()
s.introduce()
s.learn()
```

```
I am a person
I am learning Python
```

Task 2 Manager uses super().

Solution :

```
class Employee:
    def __init__(self, name, salary):
        self.name = name
        self.salary = salary
class Manager(Employee):
    def __init__(self, name, salary, team_size):
        super().__init__(name, salary)
        self.team_size = team_size
m = Manager("Asha", 60000, 5)
print(m.name, m.salary, m.team_size)
```

```
Asha 60000 5
```

Task 3 Method overriding.

Solution :

```
class Animal:
    def sound(self):
        print("Some sound")
class Cat(Animal):
    def sound(self):
        print("Cat meows")
Cat().sound()
```

```
Cat meows
```

Task 4 SSC and HSC examination.

Solution :

```
class SSC:
    def __init__(self, marks):
        self.marks = marks
    def examination(self, student):
        print(f"{student} scored {self.marks}% in SSC")
class HSC:
    def __init__(self, marks):
        self.marks = marks
    def examination(self, student):
        print(f"{student} scored {self.marks}% in HSC")
SSC(83).examination("Namrata")
HSC(90).examination("Namrata")
```

```
Namrata scored 83% in SSC
Namrata scored 90% in HSC
```

Task 5 Abstract Shape and Rectangle.

Solution :

```
from abc import ABC, abstractmethod
class Shape(ABC):
    @abstractmethod
    def area(self):
        pass
class Rectangle(Shape):
    def __init__(self, length, width):
        self.length = length
        self.width = width
    def area(self):
        return self.length * self.width
print("Area:", Rectangle(5, 3).area())
```

```
Area: 15
```

Task 6 Abstract class cannot be created.

Solution :

```
from abc import ABC, abstractmethod
class Shape(ABC):
    @abstractmethod
    def area(self):
        pass
try:
    Shape()
except TypeError as e:
    print("TypeError: cannot create Shape")
```

```
TypeError: cannot create Shape
```